Key Matrix Reference Layout Overview

Event Structural String	Target Physical System Behavior
`"<Shift-KeyPress-equal>"`	Captures the standard main keyboard **Plus (+)** symbol via standard physical modification.
`"<KeyPress-plus>"`	Direct filter capture for the literal plus key event sequence.
`"<KeyPress-KP_Add>"`	Target lock for the independent hardware numeric pad plus key (**Numpad +**).
`"<Control-KeyPress-plus>"`	Multi-key combination explicitly listening for standard application zoom modifications.
`"<KeyPress-Return>"`	Standard keyboard main line confirmation enter sequence.
`"<KeyPress-Escape>"`	Core abort application execution system instruction hook.

Would you like assistance **building a dynamic Python script** that prints out every single keysym option directly from your machine's active operational system environment, or do you need help **mapping a specific dual-modifier macro sequence**?

any idea on parentesis open lose dbrckets open losed curyl brakts if those are naemed

Yes, all of these symbols have specific **Keysym names** in Tkinter!

Because these symbols usually require holding down the `Shift` key on standard keyboards, Tkinter allows you to bind them in two ways: by their **direct literal symbol name** (the easiest way) or by their **explicit Shift combination**.

Here is the exact reference list for parentheses, brackets, and braces in Tkinter:

📋 Parentheses, Brackets, & Braces Reference Table

Symbol	Common Name	Direct Tkinter Keysym	Explicit Shift Combination
`(`	Parenthesis Open	`"<parenleft>"`	`"<Shift-KeyPress-9>"`
`)`	Parenthesis Close	`"<parenright>"`	`"<Shift-KeyPress-0>"`

`[`	Bracket Open	`"<bracketleft>"`	`"<KeyPress-bracketleft>"` *(No Shift)*
`]`	Bracket Close	`"<bracketright>"`	`"<KeyPress-bracketright>"` *(No Shift)*
`{`	Curly Brace Open	`"<braceleft>"`	`"<Shift-KeyPress-bracketleft>"`
`}`	Curly Brace Close	`"<braceright>"`	`"<Shift-KeyPress-bracketright>"`

💡 Pro-Tip for Tkinter Coding